Pmod OLEDrgb per SPI am Arty A7-35T

Bosch-Logo | HHN-Logo | rotierender Anker | Pong

Projekt prj_spi | Digitale Schaltungstechnik | Autor / Gruppe: noch zu ergänzen

Die fünf Bildschirme (aus den echten ROM-Daten gerendert)



sw0
Bosch-Logo

sw1
Bosch, Anker dreht sich

sw2
HHN-Logo

sw3
Pong 1 gegen 1

kein / mehrere
Wähle einen Schalter

Das echte Display ist 0,8 x 0,5 Zoll gross (Digilent) - Farben wirken dort kräftiger als am Bildschirm.

Aufgabe und Ergebnis

Schalter -> Bild (Moduswahl)

Es zählt nur eine Stellung mit GENAU einem Schalter oben. Alles andere zeigt den Hinweis.

sw3	sw2	sw1	sw0	Ergebnis
○ 0	○ 0	○ 0	○ 0	kein Schalter -> Hinweis 'Wähle einen Schalter'
○ 0	○ 0	○ 0	● 1	Bosch-Logo (Anker steht)
○ 0	○ 0	● 1	○ 0	Bosch-Logo, Anker rotiert
○ 0	● 1	○ 0	○ 0	HHN-Logo
● 1	○ 0	○ 0	○ 0	Pong, 1 gegen 1
○ 0	○ 0	● 1	● 1	mehrere Schalter -> Hinweis
○ 0	● 1	○ 0	● 1	mehrere Schalter -> Hinweis
● 1	● 1	● 1	● 1	alle oben -> Hinweis

```
VHDL (decode): when "0001" => M_BOSCH; "0010" => M_ROT; "0100" => M_HHN; "1000" => M_PONG; when others => M_MSG
```

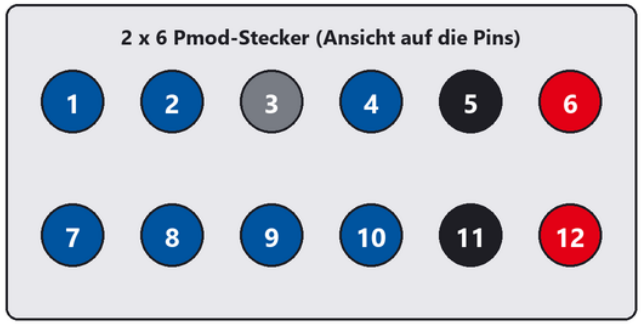
Blau gefüllt = Schalter oben (1). Der Modus wird nur bei frame_tick übernommen.

- Pmod OLEDrgb (96 x 64, SPI) an Pmod JD des Arty A7-35T
- Vier Schalter wählen das Bild: Bosch, Bosch drehend, HHN, Pong - sonst Hinweis
- Alles als Schaltung in VHDL - kein Prozessor; Simulation bestanden, Betrieb auf dem Board bestätigt

Hardware: Anschluss an JD

Anschluss: Pmod OLEDrgb an Pmod-Header JD

Der Stecker passt 1:1: Pin n des Moduls liegt auf Pin n von JD. GND auf GND (Pin 5/11), VCC auf VCC (Pin 6/12).



obere Reihe: Pin 1-6, untere Reihe: Pin 7-12

GND VCC 3,3 V Signal nicht belegt

Prüfliste vor dem Einschalten

- Modul bis zum Anschlag in JD stecken, nicht um eine Reihe versetzt (Pin 1 = CS oben links)
- bei Kabel: Pin 5+11 -> GND, Pin 6+12 -> 3V3
- nie 5 V anlegen (Pmod-Pins max. 3,4 V)

Pin	Signal	Bedeutung	JD / FPGA-Ball	Port im Design
1	**CS**	Chip Select (aktiv low)	JD1 / D4	oled_cs
2	**MOSI**	Daten FPGA -> Display	JD2 / D3	oled_mosi
3	**NC**	nicht verbunden	-	-
4	**SCK**	SPI-Takt	JD4 / F3	oled_sck
5	**GND**	Masse	GND	-
6	**VCC**	3,3 V Versorgung	3V3	-
7	**D/C**	0 = Kommando, 1 = Pixeldaten	JD7 / E2	oled_dc
8	**RES**	Reset des Controllers	JD8 / D2	oled_res
9	**VCCEN**	Schaltet Panelspannung	JD9 / H2	oled_vccen
10	**PMODEN**	Schaltet Logik-Masse	JD10 / G2	oled_pmoden
11	**GND**	Masse	GND	-
12	**VCC**	3,3 V Versorgung	3V3	-

JD ist ein Standard-Pmod mit 200-Ohm-Schutzwiderständen - für SPI mit 6,25 MHz unkritisch.
Quelle: Digilent Arty A7 Reference Manual, Abschnitt 10 (Pmod Connectors); Pmod OLEDrgb Reference Manual, Pinout Description Table.

- Pmod-Stecker passt 1:1: GND auf Pin 5/11, VCC (3,3 V) auf Pin 6/12
- JD ist ein Standard-Pmod mit 200-Ohm-Schutzwiderständen [1]
- Signale: CS, MOSI, SCK, D/C, RES, VCCEN, PMODEN [2]

Was ist ein FPGA?

FPGA-Grundbausteine

Ein FPGA besteht aus sehr vielen kleinen, frei verschaltbaren Bausteinen - Logik und Speicher.

1 LUT: Logik als Tabelle

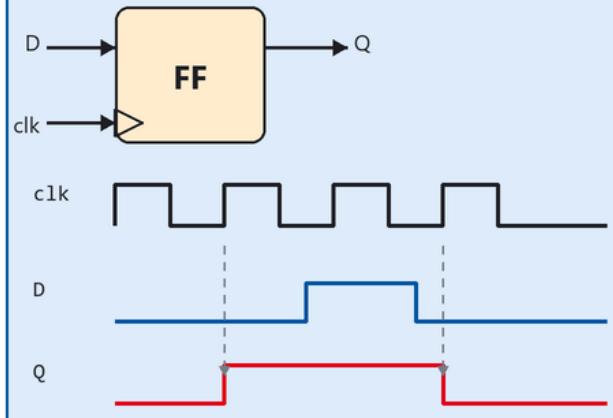
A	B	Y = A UND B
0	0	0
0	1	0
1	0	0
1	1	1



Eine LUT (Look-Up Table) ist ein winziger Speicher. Die Eingänge sind die Adresse, der Inhalt ist das Ergebnis. So kann jede Logikfunktion gebaut werden.

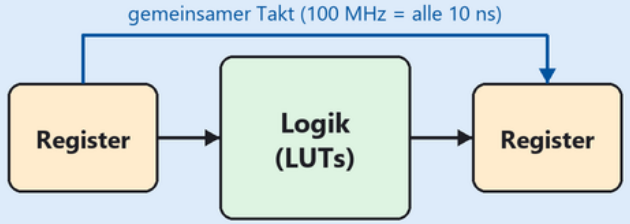
Artix-7: 6 Eingänge pro LUT
(35T: 5 200 Slices zu je 4 LUT).

2 Flip-Flop: 1 Bit Speicher



Q übernimmt D nur bei der steigenden Taktflanke und hält den Wert bis zur nächsten.

3 Synchrone Schaltung

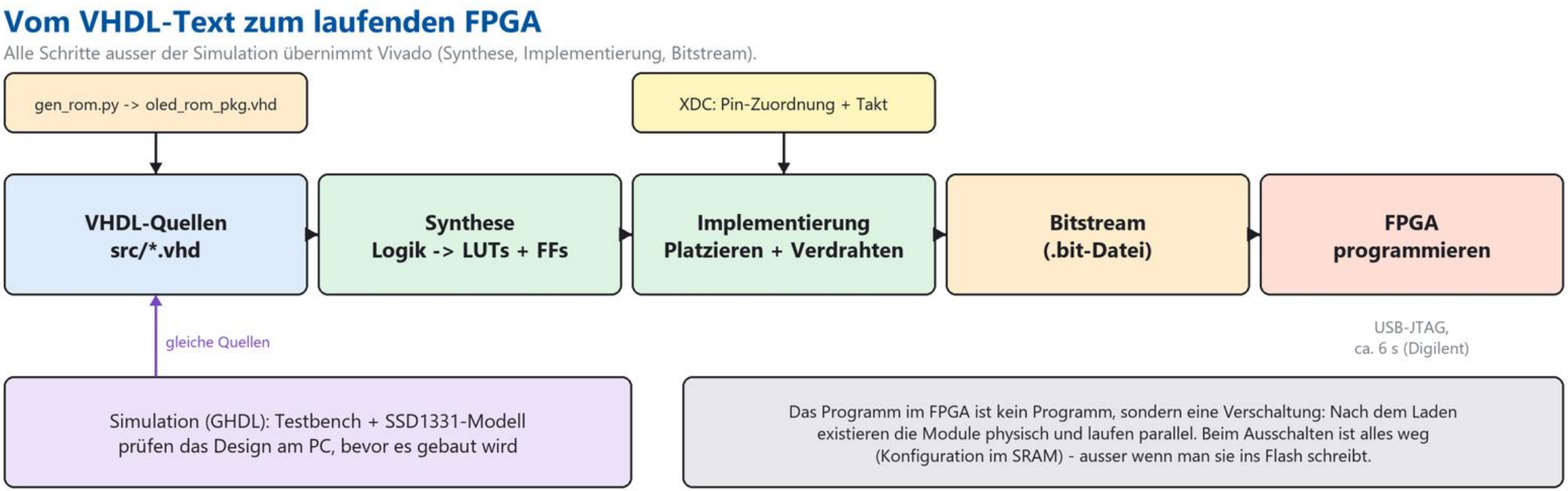


Bei jeder steigenden Flanke übernehmen alle Register gleichzeitig das Ergebnis der Logik vor ihnen. Zwischen zwei Flanken rechnet die Logik; der Takt muss so langsam sein, dass sie fertig wird (Timing).

Wichtigster Unterschied zur Software: Im FPGA wird nichts nacheinander abgearbeitet - alle Bausteine arbeiten in jedem Takt gleichzeitig.

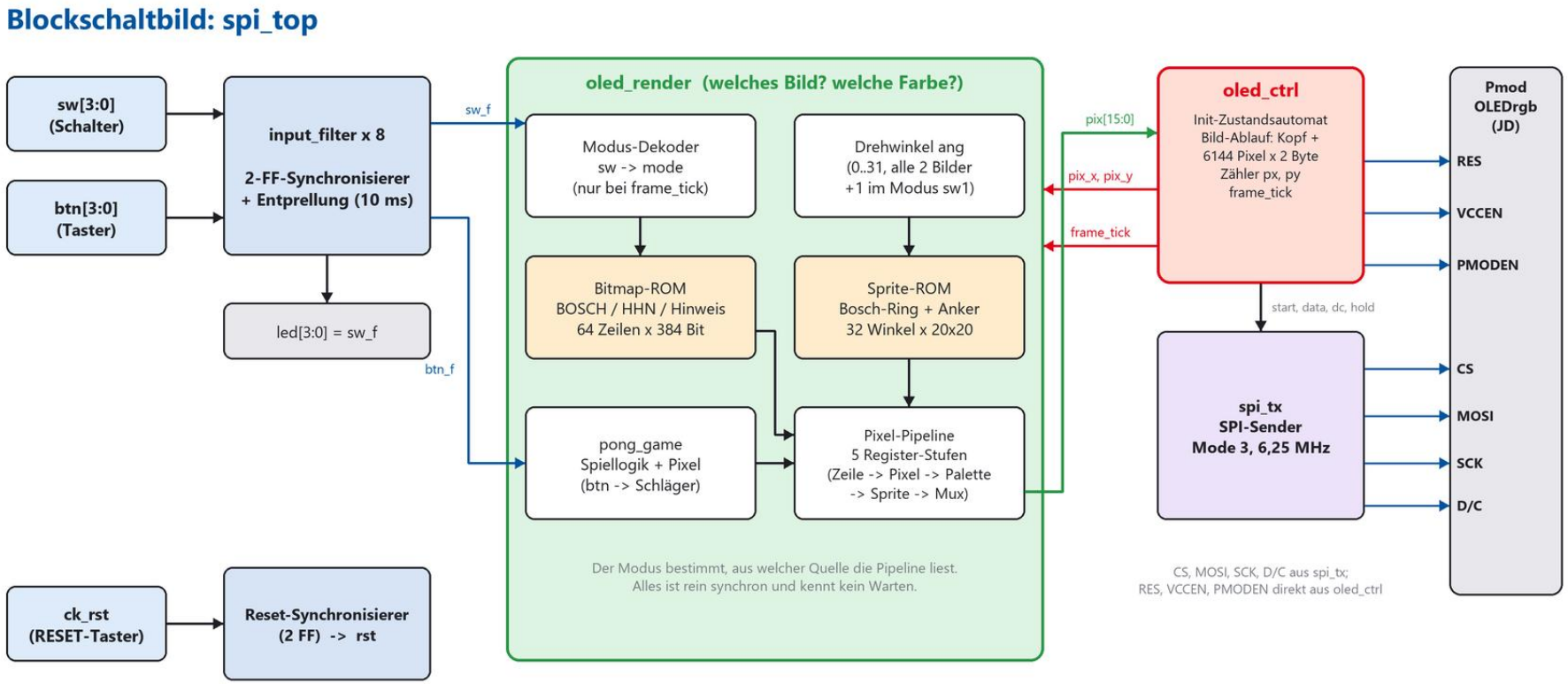
- LUT = Logikfunktion als Tabelle, Flip-Flop = 1 Bit Speicher
- Alle Bausteine arbeiten gleichzeitig im 100-MHz-Takt (10 ns)
- Man beschreibt eine Schaltung (VHDL), man programmiert nicht

Vom VHDL-Text zum laufenden FPGA



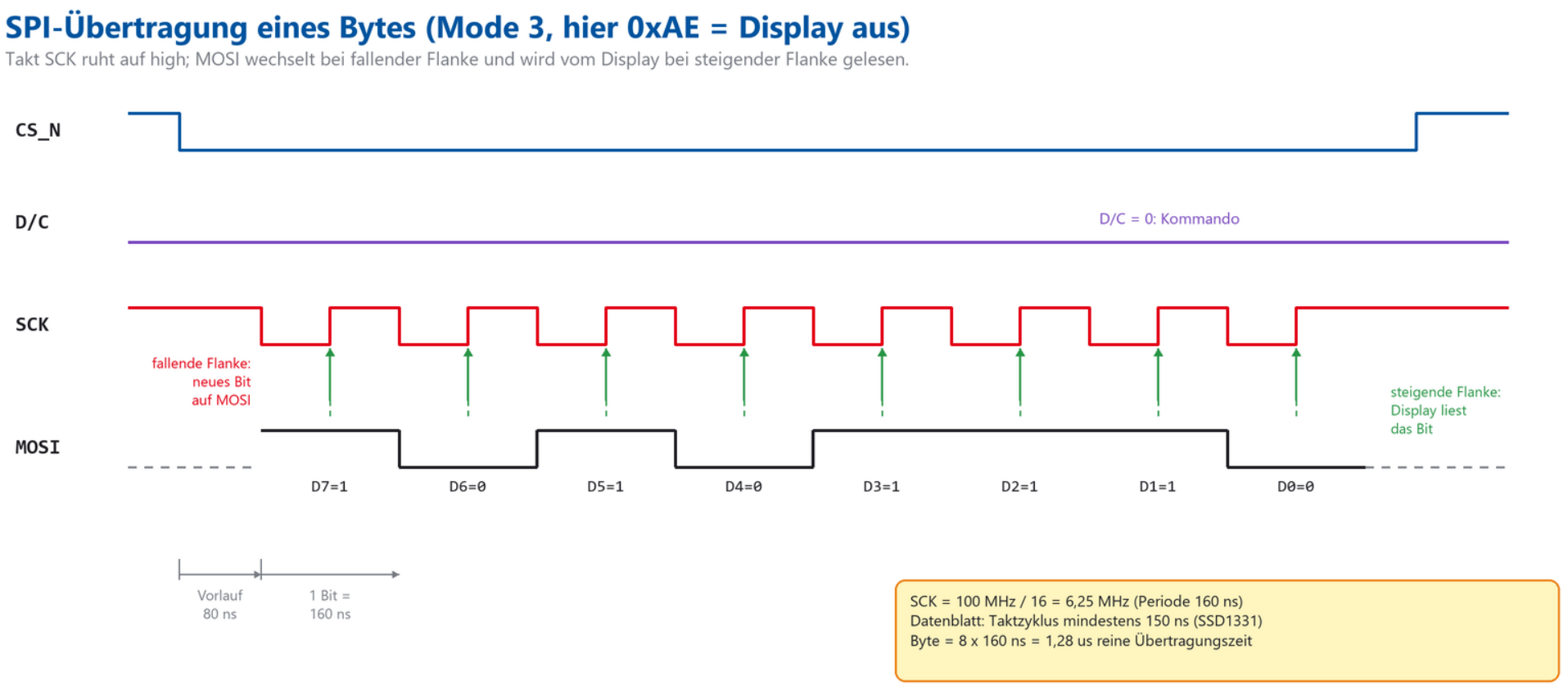
- Simulation (GHDL) prüft das Design am PC
- Vivado: Synthese, Implementierung (Pins aus der XDC), Bitstream
- Das Laden über USB-JTAG dauert ca. 6 s; die Konfiguration ist flüchtig [1]

Gesamtaufbau



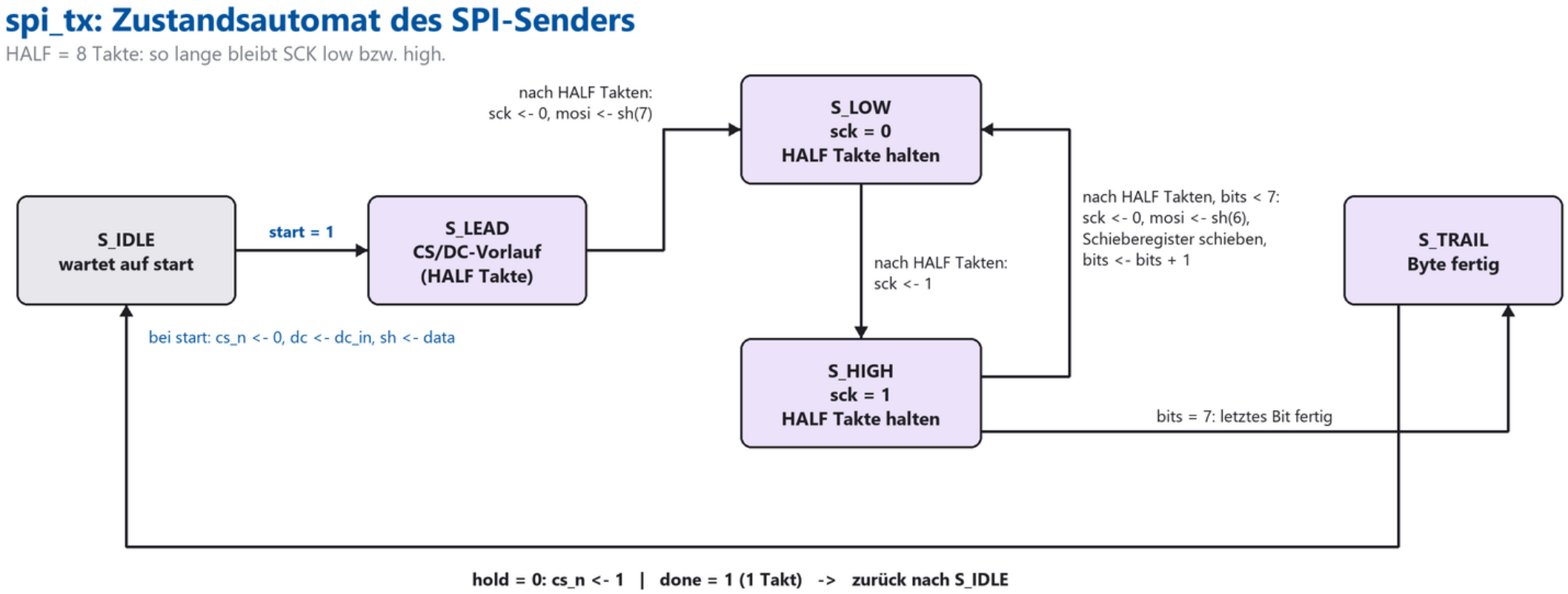
- input_filter: Sync + Entprellung für Schalter und Taster
- oled_render: Welche Farbe hat Pixel (x, y)? oled_ctrl: Ablauf und Pixelzähler
- spi_tx: sendet jedes Byte seriell an das Display

SPI Mode 3 - ein Byte



- SCK ruht auf 1; Bit wechselt bei fallender, wird bei steigender Flanke gelesen
- SCK = 100 MHz / 16 = 6,25 MHz (Datenblatt: min. 150 ns Periode) [2]
- D/C: 0 = Kommando, 1 = Pixeldaten

SPI-Sender spi_tx

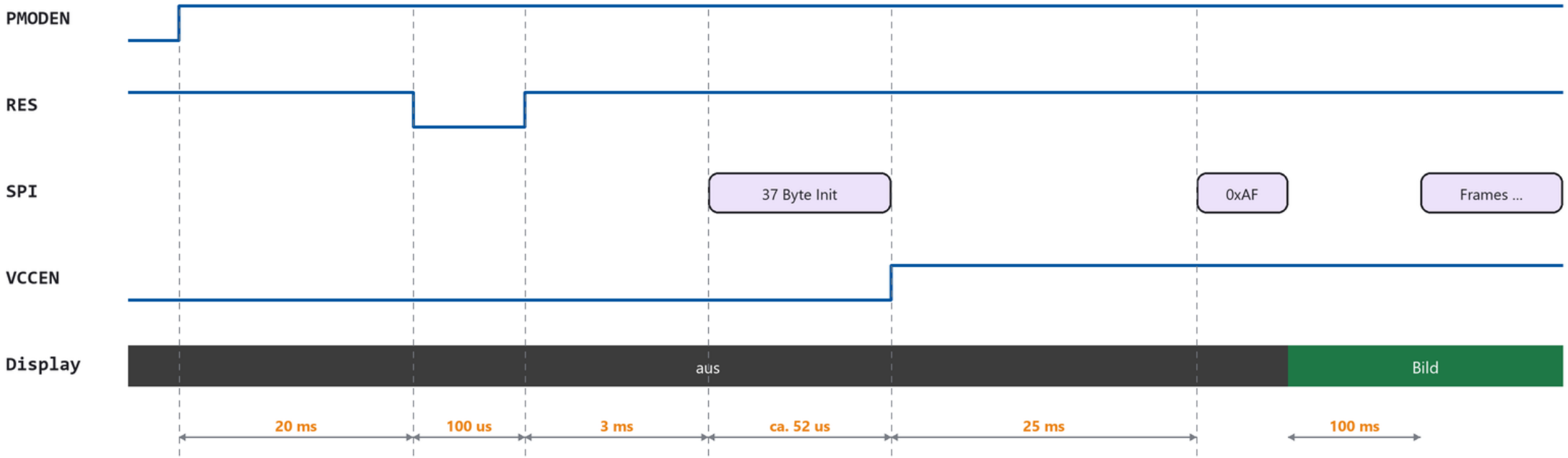


- Fünf Zustände: IDLE, LEAD, LOW, HIGH, TRAIL
- hold = 1 hält CS für den ganzen Pixelstrom auf 0
- done meldet einen Takt lang: Byte fertig

Display einschalten

Einschaltreihenfolge des Displays

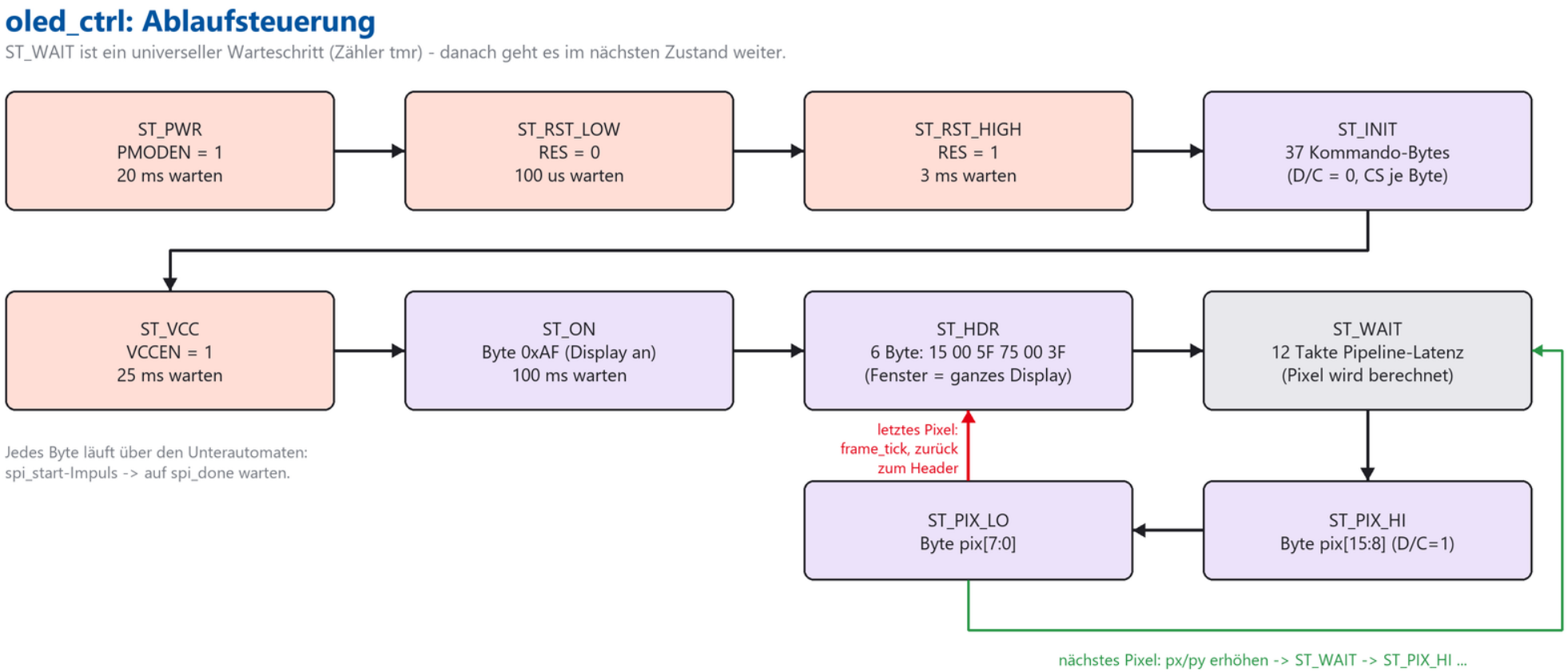
Nicht maßstäblich. Die Zeiten entsprechen der Digilent-Vorgabe (Pmod OLEDrgb Reference Manual) bzw. dem Code.



Reihenfolge ist wichtig: Erst die Logik-Versorgung (PMODEN), dann Reset, dann Konfiguration, ERST ZULETZT die hohe Panelspannung (VCCEN).

- Reihenfolge: PMODEN, Reset, Konfiguration, dann erst VCCEN und Display-ein [2]
- 37 Konfigurations-Bytes (Remap, Kontrast, Ströme ...)
- Zeiten: 20 ms, 100 µs, 3 ms, 25 ms, 100 ms

Ablaufsteuerung oled_ctrl

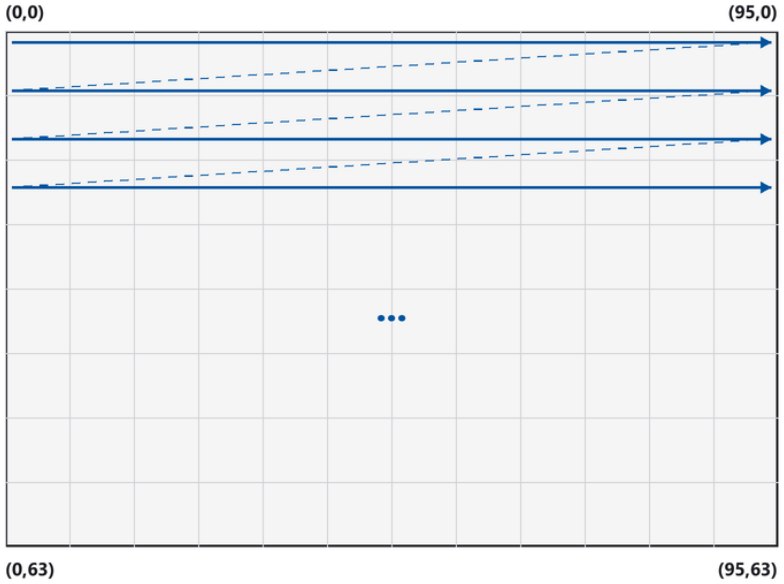


- Zustandsautomat: Strom, Reset, Init, VCC, Display ein, dann Bild-Schleife
- Pro Pixel: 12 Takte warten (Pipeline), Byte hi, Byte lo
- Nach dem letzten Pixel: frame_tick und zurück zum Kopf

Ein Bild übertragen

Wie ein Bild zum Display kommt

96 x 64 Pixel = 6144 Pixel, jeder Pixel = 2 Byte -> 12 288 Datenbytes pro Bild.



x: 0..95 (schnell) y: 0..63 (langsam) - Zeile für Zeile, links -> rechts, oben -> unten

Datenstrom über SPI pro Bild

Kopf: 15 00 5F | 75 00 3F (D/C = 0, Kommandos)

0x15 = Spaltenbereich 0..95 0x75 = Zeilenbereich 0..63 -> Fenster = ganzes Display

Pixeldaten (D/C = 1, CS bleibt über das ganze Bild low)

P(0,0).hi | P(0,0).lo | P(1,0).hi | P(1,0).lo | ... | P(95,63).hi | P(95,63).lo

Das Display zählt die Adresse selbst hoch (Zeile voll -> nächste Zeile),
der FPGA sendet nur Pixel in der richtigen Reihenfolge.

frame_tick (1 Takt) -> neuer Modus, neuer Winkel, Pong-Schritt
Danach beginnt sofort das nächste Bild mit dem Kopf.

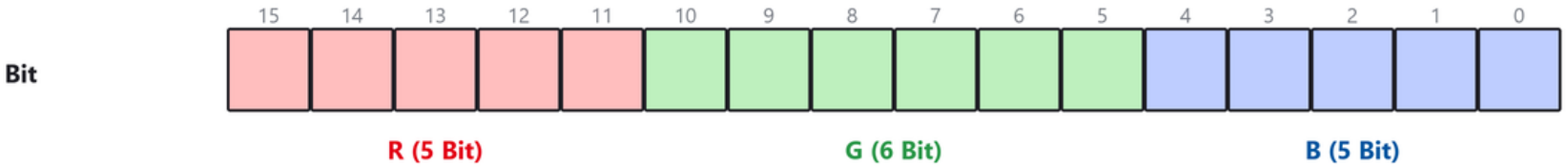
Zeit: 1 Byte ca. 1,4 us -> 1 Pixel ca. 2,9 us (inkl. 12 Takte Wartezeit)
6144 Pixel ca. 18 ms -> rechnerisch ca. 55 Bilder pro Sekunde.

- 96 x 64 = 6144 Pixel x 2 Byte = 12 288 Datenbytes pro Bild
- Kopf 15 00 5F 75 00 3F legt das Fenster fest; das Display zählt die Adresse selbst
- Zeile für Zeile, links nach rechts, oben nach unten

Farbe: RGB565

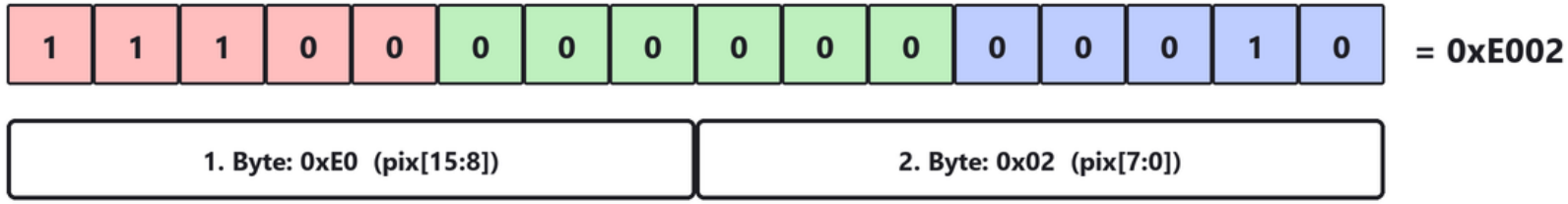
Ein Pixel = 16 Bit (RGB565)

5 Bit Rot, 6 Bit Grün, 5 Bit Blau; wird als zwei Bytes gesendet (zuerst das hohe Byte).



Beispiel Bosch-Rot (227, 0, 21):

`R = 227 / 8 = 28 = 11100` `G = 0 / 4 = 0 = 000000` `B = 21 / 8 = 2 = 00010`



Weiss = 0xFFFF (alle Bits 1) Schwarz = 0x0000 - Pong und der Hinweis nutzen nur diese beiden Werte.

Der Controller wurde mit Remap 0x72 auf 65k-Farben (RGB-Reihenfolge) eingestellt (siehe Init-Sequenz).

- 5 Bit Rot, 6 Bit Grün, 5 Bit Blau = 16 Bit pro Pixel
- Bosch-Rot (227, 0, 21) = 0xE002; Weiß 0xFFFF; Schwarz 0x0000
- Erst das obere, dann das untere Byte

Zeitbedarf eines Bildes

Zeitlicher Ablauf eines Bildes

Nicht maßstäblich - es zählt die Reihenfolge der Schritte.

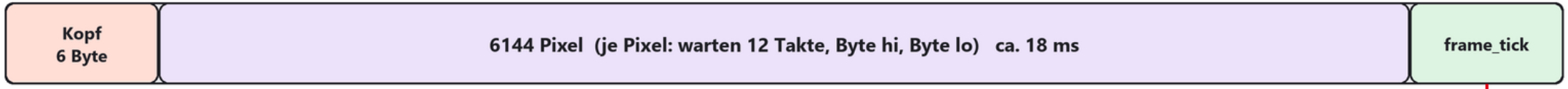
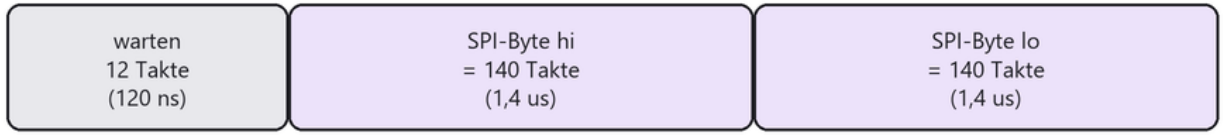


Bild n

Ein Pixel im Detail:



Summe ca. 294 Takte = 2,94 us pro Pixel

Bei frame_tick (1 Takt):

- mode <- decode(sw)
- ang <- ang + 1 (alle 2 Bilder, nur sw1)
- Pong: Schläger bewegen, Ball ein Schritt, Punkte, Aufschlag-Zähler

Rechenbeispiel: 6144 x 2,94 us = 18,06 ms (+ 6 Kopf-Bytes = 8,4 us)
Bilder pro Sekunde = 1 / 18,07 ms = ca. 55 Hz

Die Zeit ist fast nur SPI-Übertragung. Würde man SCK auf 12,5 MHz erhöhen, würde es doppelt so schnell - das erlaubt der Controller laut Datenblatt aber nicht (min. 150 ns Taktzyklus).

- Pixel: 12 + 140 + 140 Takte = ca. 294 Takte = 2,94 µs
- 6144 Pixel ca. 18 ms, rechnerisch ca. 55 Bilder pro Sekunde
- Modus, Winkel und Spiel ändern sich nur bei frame_tick

Die fünf Bildschirme

Die fünf Bildschirme (aus den echten ROM-Daten gerendert)



sw0
Bosch-Logo

sw1
Bosch, Anker dreht sich

sw2
HHN-Logo

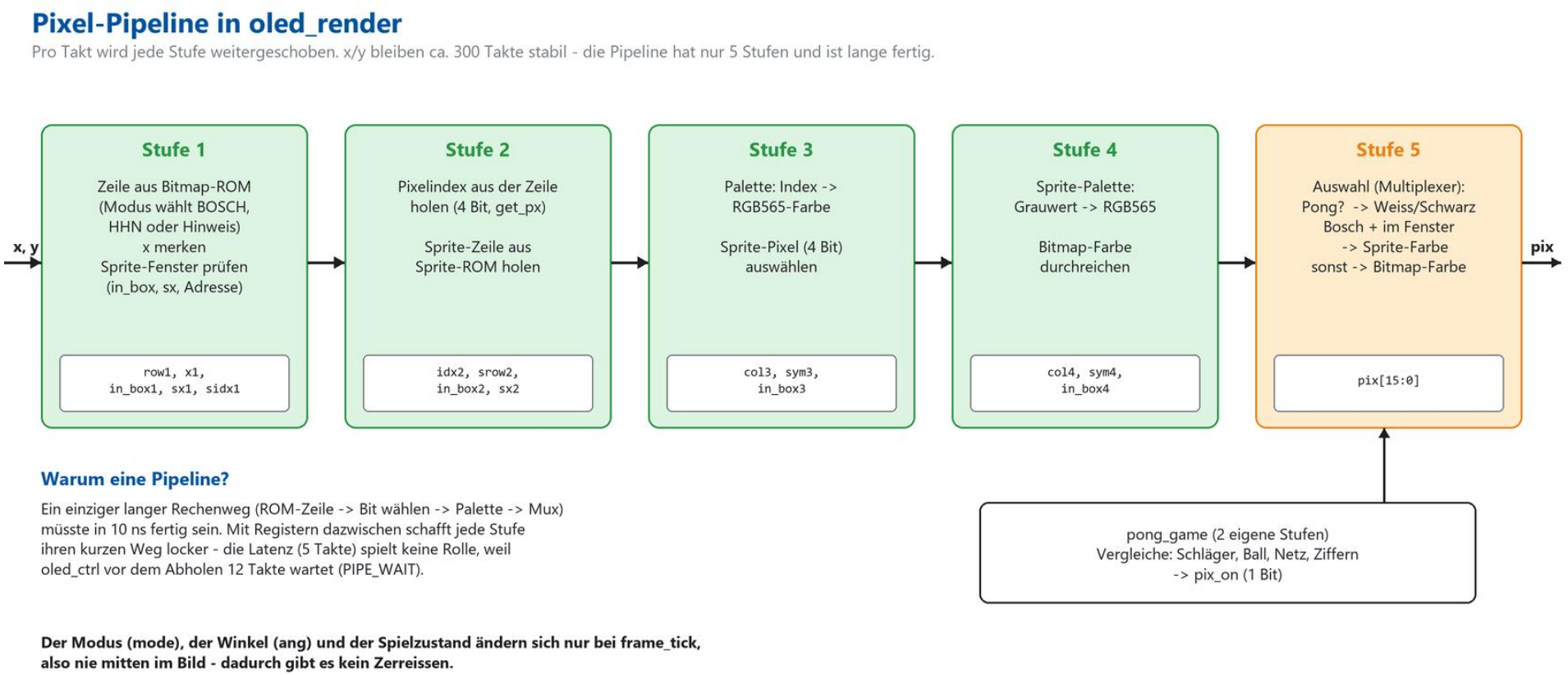
sw3
Pong 1 gegen 1

kein / mehrere
Wähle einen Schalter

Das echte Display ist 0,8 x 0,5 Zoll gross (Digilent) - Farben wirken dort kräftiger als am Bildschirm.

- Bosch: Symbol links (Sprite), Schriftzug rechts (Bitmap)
- HHN-Logo nach Vorlage nachgezeichnet (oder eigenes PNG einsetzen)
- Hinweis und Pong in Weiß auf Schwarz

Pixel-Pipeline



- Fünf Registerstufen: Zeile, Pixel, Palette, Sprite, Auswahl
- Kurze Wege zwischen den Registern: hoher Takt ohne Probleme
- Latenz 5 Takte, Wartezeit 12 Takte: keine Handshake-Logik nötig

Bilder im ROM

Bitmap-ROM: so liegt ein Bild im FPGA

Jede Zeile ist ein 384-Bit-Wort: 96 Pixel x 4 Bit Palettenindex. Pixel 0 steht ganz links (MSB).

Beispiel: Zeile y = 32 des Bosch-Bitmaps

x=0 idx=0	x=1 idx=0	x=2 idx=0	x=3 idx=0	...	x=31 idx=0	x=32 idx=C	x=33 idx=F	...	x=95 idx=0

als Hex-Zeichen im VHDL (je Pixel eine Stelle):

```
00000000000000000000000000000000CFFFFFFFFC20CFFA
00008FFE00039EFFFFF41EFF8000000003FFFFFFFFFB000
```

Pixel x liegt in den Bits (383 - 4x) bis (380 - 4x) -> get_px() prüft für x = 0..95, welcher Treffer es ist.

Palette (16 Einträge je Bild, RGB565):

idx 0 0xFFFF	idx 1 0xFFFF	idx 2 0xFF9E	idx 3 0xFEDB	idx 4 0xFE18	idx 5 0xF576

Speicherbedarf (aus gen_rom.py):

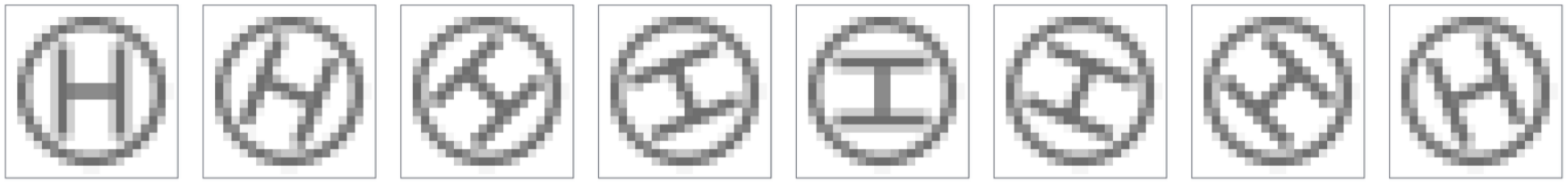
3 Bitmaps (Bosch, HHN, Hinweis): 3 x 64 x 384 Bit = 73 728 Bit
Sprite-ROM (32 Winkel x 20 Zeilen x 80 Bit) = 51 200 Bit
Das passt bequem in den Artix-7 (225 KB Block-RAM, zusätzlich LUT-ROM).
Die Bilder ändert man nie von Hand: python scripts/gen_rom.py erzeugt src/oled_rom_pkg.vhd neu.

- Pixel = 4-Bit-Index in eine 16-Farben-Palette (RGB565)
- Eine Zeile = 384 Bit; 3 Bilder = 73 728 Bit
- gen_rom.py zeichnet, verkleinert, reduziert auf 16 Farben und schreibt VHDL

Drehender Anker: 32 Sprites

Bosch-Symbol: 32 vorgerenderte Dreh-Sprites

Der Anker (H) hat nach 180 Grad wieder dieselbe Form - 32 Schritte x 5,625 Grad reichen für die volle Drehung.



k = 0 (0.0 Grad) | k = 4 (22.5 Grad) | k = 8 (45.0 Grad) | k = 12 (67.5 Grad) | k = 16 (90.0 Grad) | k = 20 (112.5 Grad) | k = 24 (135.0 Grad) | k = 28 (157.5 Grad)



Bosch-Bild: Schriftzug kommt aus dem Bitmap, das rote Quadrat (20 x 20 Pixel, Ecke (1,22)) aus dem Sprite.

Sprite-Fenster

Liegt der aktuelle Pixel (x, y) im Fenster, ersetzt die Sprite-Farbe die Bitmap-Farbe:

```
sx = x - 1      sy = y - 22      (0..19)
Zeile im Sprite-ROM = ang * 20 + sy
Pixel = Bits (79 - 4 sx) .. (76 - 4 sx)
Farbe = SYM_PAL(Pixel)   (16 Grauwerte von Weiss bis Grau)
```

sw0: ang = 0 (H steht aufrecht)
sw1: ang zählt alle 2 Bilder hoch (0..31, dann wieder 0)

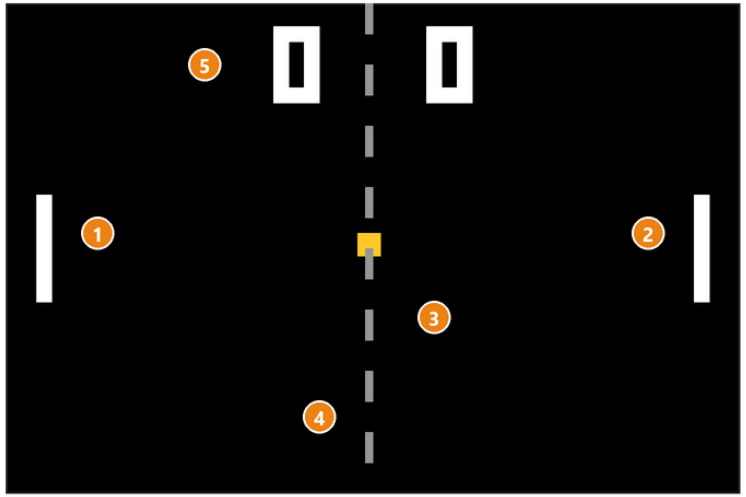
Vorteil: keine Multiplikation / kein Sinus im FPGA, Kanten sind sauber geglättet. Nachteil: nur die 32 festen Winkel.

- Ring + Anker 32 mal in 5,625 Grad Schritten vorgerendert (20 x 20 Pixel)
- Sprite-Fenster ersetzt im Bosch-Bild die Bitmap-Farbe
- Keine Multiplikation im FPGA, saubere Kanten

Pong: Spielfeld

Pong: Spielfeld und Abmessungen (alle Angaben in Pixeln)

Das Feld ist das Display selbst: 96 x 64, Ursprung (0,0) oben links.



Wände: oben (y = 0) und unten (y = 63) - der Ball prallt ab.
Tor: Verlässt der Ball links (x <= 0) oder rechts (x + 3 >= 96)
das Feld, bekommt der Gegner einen Punkt.

1 Linker Schläger
x = 4..5 (2 breit), 14 hoch, Start y = 25..38
btn0 hoch, btn1 runter, je 3 px pro Bild

2 Rechter Schläger
x = 90..91, 14 hoch, Start y = 25..38
btn2 hoch, btn3 runter

3 Ball
3 x 3 Pixel, Start bei (46,30)
Geschwindigkeit vx = +-2 .. +-4, vy = -2..+2 px/Bild

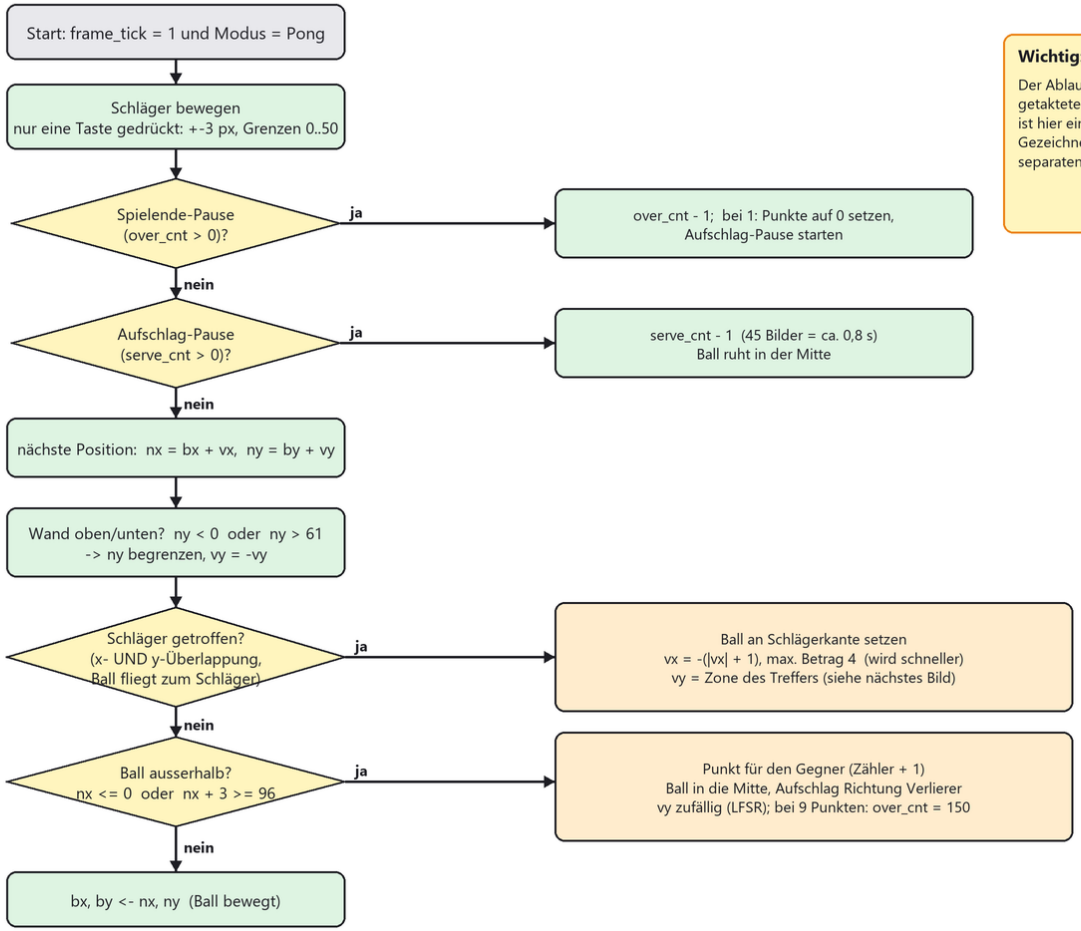
4 Netz
x = 47, 4 Pixel an / 4 Pixel aus (nur Optik)

5 Punktestand
3 x 5 Ziffern, doppelt vergrößert (6 x 10 px)
links ab x = 35, rechts ab x = 55, y = 3

- Feld = Display: 96 x 64; Schläger 2 x 14, Ball 3 x 3
- btn0/btn1 linker, btn2/btn3 rechter Schläger
- Ball wird mit jedem Treffer schneller (max. 4 Pixel pro Bild)

Pong: Ablauf pro Bild

Pong: was bei jedem frame_tick passiert (ca. 55 mal pro Sekunde)



Wichtig:

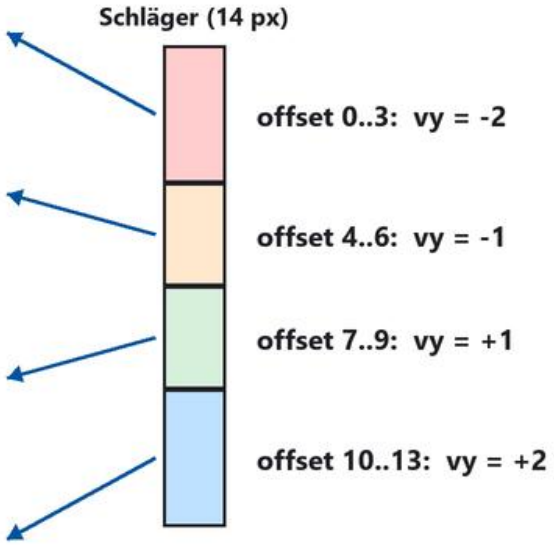
Der Ablauf ist ein einziger getakteter Prozess. Ein 'Bild' ist hier ein Spielschritt. Gezeichnet wird in einer separaten Pipeline.

- Ein Spielschritt pro frame_tick (ca. 55 pro Sekunde)
- Wand, Treffer (x- und y-Überlappung), Tor
- Zufälliger Aufschlagwinkel aus einem LFSR

Pong: Abprallwinkel

Pong: Ablenkwinkel hängt vom Treffpunkt ab

offset = (ny + 1) - Schlägerposition (Ball-Mitte relativ zur Schlägeroberkante)



Der Ball kommt von rechts, vx wechselt das Vorzeichen
(und wächst um 1 bis max. 4).

Obere Hälfte -> Ball fliegt nach oben (vy negativ)
Untere Hälfte -> Ball fliegt nach unten (vy positiv)
Aussen steiler (+-2), innen flacher (+-1).

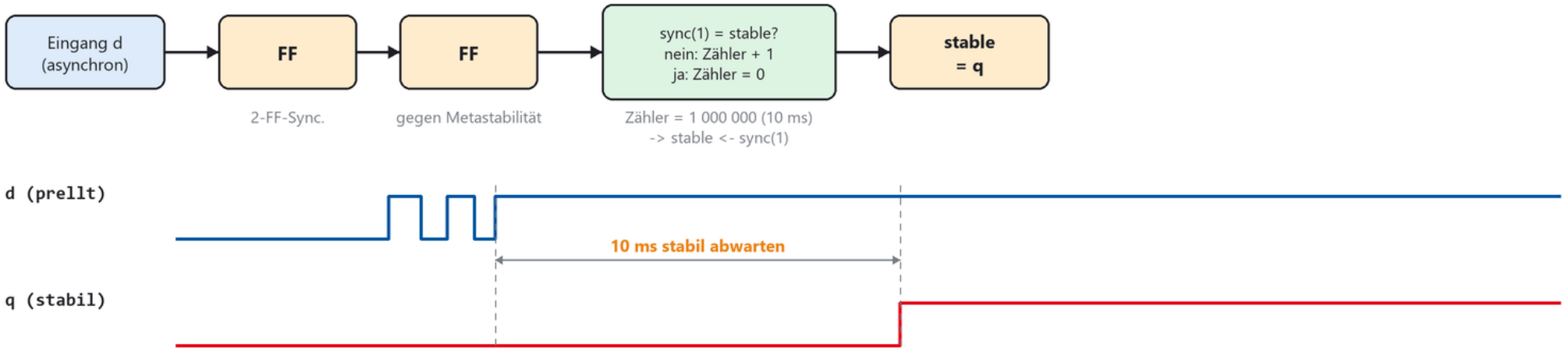
Damit kann der Spieler den Ball gezielt lenken;
eine Ablenkung von 0 gibt es bewusst nicht -
der Ball fliegt nie waagerecht.

- Vier Zonen auf dem Schläger: vy = -2, -1, +1, +2
- Außen steiler, innen flacher; nie waagerecht
- So lässt sich der Ball gezielt lenken

Eingänge entprellen

Entprellung: input_filter

Mechanische Schalter prellen einige Millisekunden. Der Filter übernimmt einen neuen Pegel erst, wenn er 10 ms lang stabil war.



Wichtig für das Design: Alle Schalter und Taster laufen durch input_filter (8 Stück).
Die Taster liegen im Ruhezustand auf 0 und liefern 1, wenn gedrückt (Arty A7). Der RESET-Taster ist umgekehrt (aktiv low) und wird nur durch einen 2-FF-Synchronisierer geführt.

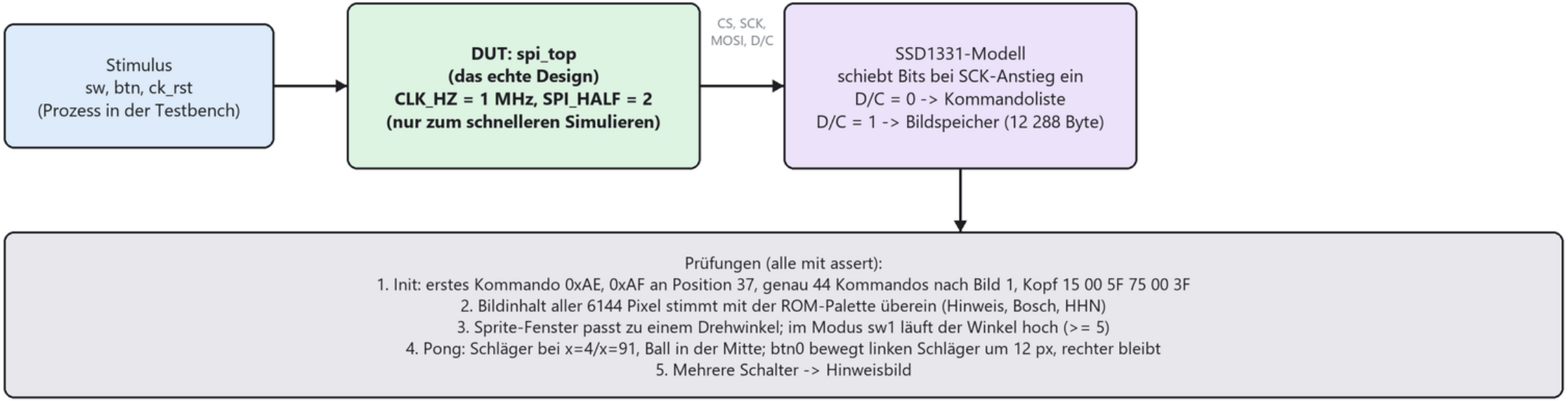
Quelle zu den Tastern: Digilent Arty A7 Reference Manual, Abschnitt 9 (Basic I/O).

- 2 Flip-Flops gegen Metastabilität, Zähler für 10 ms
- Taster: Ruhe 0, gedrückt 1; RESET: aktiv low [1]
- Reset wird einsynchronisiert und initialisiert das Display neu

Simulation und Tests

Simulation: tb_spi_top (GHDL)

Ein Software-Modell des Displays liest die SPI-Signale mit und prüft den Inhalt Bit für Bit.



- SSD1331-Modell in der Testbench liest SPI mit und baut das Bild nach
- Geprüft: Init-Bytes, alle Pixel der Bilder, Drehwinkel, Pong-Schläger
- Nicht geprüft: Timing auf dem Chip, Pins, Display-Orientierung

Fazit und Ausblick

- Ergebnis: vier Anwendungen plus Hinweisbild auf einem 96 x 64 Pixel Display, komplett in VHDL
- Gelernt: SPI, Display-Initialisierung, Pipeline, Zustandsautomaten, Testbench mit Gerätemodell
- Ideen: eigene Bilder (assets/), Siegertext in Pong, höherer SPI-Takt, Text-Zeichensatz

Quellen

- [1] Digilent: Arty A7 Reference Manual - digilent.com/reference/programmable-logic/arty-a7/reference-manual
- [2] Digilent: Pmod OLEDrgb Reference Manual - digilent.com/reference/pmod/pmodoledrgb/reference-manual
- [3] Solomon Systech: SSD1331 Datasheet (in [2] verlinkt)
- [4] Digilent: Arty-A7-35-Master.xdc (Pin-Namen)
- Zugriff: Seiten über Internet-Archive-Abbild (07.11.2024), direkter Abruf lieferte HTTP 403